NOTEFUSION · PYTHON TRANSCRIPTION PIPELINE

Transcription accuracy: the plan the measurements actually support

Audited against the code and the committed accuracy baseline — 22 August 2026

Source under audit **docs/notefusion-consolidated.txt** · Baseline **server/tests/accuracy/baseline.json**, corpus **0146e7b5cb03b287** at **38 cases**

CONTENTS

Context

- Verdict on the document, on accuracy merit
- The measured defect list

Phase 1 — accuracy available now, no new model required

- 1a. Lift the pyin refinement cap — largest immediate accuracy gain
- 1b. Fold the tempo fallback into the notatable band
- 1c. Fuse fragments across a contour-continuous gap
- 1d. Instantiate the Basic Pitch model once

Phase 2 — make the result measurable (critical path for Phase 3)

- 2a. Real recordings
- 2b. Meter and metrical level

Phase 3 — engine upgrade, chosen head-to-head

- 3a. Segmentation bake-off (the defect)
- 3b. Neural downbeat and meter (the capability gap)
- 3c. Ensemble the confidence signal
- 3d. Source separation, gated
- 3e. Fine-tuning, gated and validated on real audio only

Phase 4 — what accuracy needs from the new platform

Verification

Context

`docs/notefusion-consolidated.txt` collects three AI-generated advice sets on improving the Python transcription pipeline. Audited against the code and the committed baseline (`server/tests/accuracy/baseline.json`, `report.json`), its **diagnosis is inverted**: it spends its top two priorities on *pitch tracking*, which the harness says is close to solved (`solfa.degree_accuracy 0.943`, `syllable_accuracy 0.938`), and never identifies the defect that 11 of 38 cases actually fail on.

The objective here is **transcription accuracy**. Infrastructure is treated as an enabler: the worker is moving to a platform with real compute and RAM, so torch-class models are admissible and the plan is ordered by expected accuracy gain rather than by cost.

Three things the audit established that reorder the document's priorities:

1. **The pipeline is currently running its *worse* configuration on long clips.** `settings.pyin_max_seconds` is 60 s while `max_clip_seconds` allows 180 s, so any clip over a minute **skips pitch refinement entirely** and keeps raw Basic Pitch pitches. That discards `pyin_semitone`, the largest single measured gain in the whole pipeline (`degree_accuracy` 0.831→0.917, `syllable_accuracy` 0.828→0.912, `key.exact` 0.763→0.868). The cap exists *only* because pyin is ~0.22× realtime on a single-worker queue. This is the one place a compute budget directly costs accuracy, and the platform move fixes it with a config change.
2. **The remaining error is note *segmentation*, not pitch.** `report.json` is almost perfectly bimodal: **29 of 38 cases have `n_est == n_ref` exactly**, while 9 vibrato/drift cases run 20–47 detected events against 11 real ones. Every one of those 9 is *also* a tempo failure. One root cause, three symptoms — note count, ghost syllables, tempo.
3. **Four of the document's recommendations have already been built, measured and struck**, each with a written post-mortem in the code. Evidence, not compute, is what settled them; more compute does not reopen them.

Verdict on the document, on accuracy merit

ITS RECOMMENDATION	VERDICT
Replace pyin with FCPE / torchcrepe / CREPE / PENN	**Not the defect.** Pitch labelling is at 0.943 degree accuracy; segmentation is what fails. `pyin_semitone` is precisely the stage it proposes to remove. Worth testing only as an *ensemble* member for the weak confidence signal (Phase 3c).
madmom `RNNDownBeatProcessor` for downbeats/meter	**Right goal, wrong package.** madmom needs `numpy<1.24` and uses the removed `np.float` / `np.int` aliases, against this project's Python 3.12.3 / numpy 2.4.6 — a per-environment conflict, so it is only usable in an isolated sidecar. Prefer a modern torch downbeat tracker with no such conflict (Phase 3b).
HMM chord recognition to find the key	**Low headroom, wrong material.** `key.exact 0.895`, `key.confidence_auc 0.949`, 4 cases left — and the input is monophonic humming, which has no chords to recognise.
"Preserve continuous pitch — the pipeline is integer-only"	**Factually wrong about the code, and its literal form was measured and lost.** `Event.bend_semitones`, `Event.f0`, `pyin_midi`, `voiced_prob` already carry continuous f0. Bend-corrected pitch is a *worse* notated pitch: 135 events closer to truth, 164 further. **Its unstated kernel — derive note boundaries from the contour — is the #1 defect (Phase 1c).**

ITS RECOMMENDATION	VERDICT
Enable `f0_merge`; lower `PYIN_MIN_VOICED_PROB` ("quick wins")	**Both struck on measurement.** `f0_merge`: `solfa.ghost_count` 0.105 *worse*, past the harness's 0.02 tolerance, for one accidental gained. Voiced-prob floor: a coin flip between two vibrato cases (0.333→0.875 against 0.600→0.000).
Global model singletons instead of per-call load	**Correct.** `predict()` is called with a *path* inside `_transcribe_wav`, so the ONNX session is rebuilt every job. Adopted (Phase 1d).
PyAV instead of `subprocess` ffmpeg	Rejected — not on the error budget. `_find_ffmpeg_cmd` / `_ensure_wav_path` work and ffmpeg is already an `Aptfile` dep.
Contour caching by audio hash	**Violates the privacy contract**, which compute does not change: audio is deleted the moment processing ends (TRD par. 9, `app/jobs/queue.py`).
MT3 / YourMT3+ / Transkun / SheetSage as a second engine	**Now admissible, and aimed at the right layer** (segmentation). Adopt by head-to-head measurement on real audio, not on reputation (Phase 3a).
Demucs v4 source separation	Cheap to evaluate once compute exists, but gated on real-corpus evidence that accompaniment is a live failure mode — the product's material is a monophonic line.
Train a correction model on 10k–100k synthetic mixtures	**This is the exact loop the four strikes came from.** Synthetic vibrato is a clean sinusoid; human vibrato is not. Permissible only as augmentation, validated on real clips (Phase 3e).
"Don't make Basic Pitch perfect through post-processing"	**Its one genuinely good instinct**, and it points at segmentation — not at pitch, where it spent its effort.

The measured defect list

#	DEFECT	EVIDENCE	WHERE
1	Long clips skip pitch refinement entirely	`pyin_max_seconds` 60 s vs `max_clip_seconds` 180 s	`config.py:36-46`
2	Un-notatable tempo accepted on the fallback path	`fast_sixteenths` 345 vs true 88; `vibrato_rate_k20` 398 vs true 96 — a value the code's own docstring calls "nonsense"	`transcriber.py:1965-2015`
3	Vibrato fragmentation survives every merge stage	9 cases at 2–4× over-detection; drives `n_est` 16.03 vs `n_ref` 11.18 and `ghost_count` 4.95	`transcriber.py:1229-1403`
4	Metrical level never plumbed through	The code says so at `:2195`; 11 of 38 tempi wrong	`transcriber.py:1910-2198`
5	Meter is unscoreable, so no detector can be judged	No `meter` entry in `metric_namespaces`; `Case` has no `time_signature` field	`metrics.py`, `corpus.py:89-105`
6	Per-note confidence barely discriminates	`conf.auc 0.676`, `conf.separation 0.032` — against `key.confidence_auc 0.949`	`transcriber.py:1530`
7	ONNX session rebuilt per request	`predict(str(wav_path), ICASSP_2022_MODEL_PATH, …)`	`transcriber.py:2076-2081`

Phase 1 — accuracy available now, no new model required

These four need no measurement to *justify* (each is a correctness or capability fix, not a tuned constant), and they set the honest bar any new engine in Phase 3 must beat.

1a. Lift the pyin refinement cap — largest immediate accuracy gain

`server/app/config.py:36-46`. Raise `pyin_max_seconds` to at least `max_clip_seconds` (180) so every accepted clip gets refinement, and rewrite the docstring to record that the cap was a latency guard on a single-worker CPU queue rather than an accuracy judgement.

On the new platform, pair it with `inference_workers` > 1 (`config.py:16`) and drop `WEB_CONCURRENCY: 1`. Verify the queue still deletes audio immediately per job — the concurrency change touches `app/jobs/queue.py`, which owns that contract.

Measure the actual gain by adding one long-clip case to the corpus (all 38 current cases are short, so today's baseline cannot see this defect at all).

1b. Fold the tempo fallback into the notatable band

`transcriber.py`, `_estimate_tempo_and_beats` (~1965–2015).

`_grid_bpm` refuses any candidate outside `_MIN_NOTATED_BPM` (40) … `_MAX_NOTATED_BPM` (200), and its comment calls that top bound "load-bearing". When `_grid_bpm` *declines*, the very next line returns librosa's `tracked_bpm` checked only against `_MIN_BPM` (20) … `_MAX_BPM` (400). **The same function applies two different notatability rules on two paths through itself**, so a tempo the grid stage would have refused as absurd is returned unchanged.

Fix: halve/double `tracked_bpm` (and `_onset_bpm`'s fallback) into `[_MIN_NOTATED_BPM, _MAX_NOTATED_BPM]` before returning. Reuse the bounds already at `transcriber.py:1821-1822`; add no new constant.

Predicted, stated so it can falsify the change:

- `fast_sixteenths_c_major` 345 → 172.5 (|err| 257 → 84.5)
- `vibrato_rate_k20` 398 → 199 (|err| 302 → 103)
- `aggregate.tempo.abs_error` **21.26 → ~11.5**; `rel_error` **0.229 → ~0.123**
- `tempo.octave_error` 0.0 → ~0.053, which is *correct* reporting: both become clean 2.0 metrical slips inside MIREX ±4%. Per the `tempo_grid` docstring an octave-slipped tempo is harmless — `sec_to_beats` and the preview's `beat_sec` are reciprocals, so the engraved rhythm stays right and only the printed BPM differs from what a human taps.

Two cases carry ~69% of the aggregate tempo error, and the bound that catches them is already defined in the file.

1c. Fuse fragments across a contour-continuous gap

`_merge_onsets` (~1229) plus the contour from `_pyin_contour` / `_refine_pitches` (~1409–1530).

This is the document's §4 kernel and it is a **new axis** — distinct from all four recorded strikes, which tested tolerance in *pitch* (semitone-rounded, then f0-bend), tolerance in *time* (the overlap window), and a *gate on the median*. None asked the contour whether a gap is real.

`pyin_semitone` already relabels a wobble run to one pitch, so surviving fragments are pitch-identical; `_resolve_melody_overlaps` already fuses the *overlapping* ones (`WOBBLE_SEMITONES`, `WOBBLE_MIN_OVERLAP_FRACTION`). Yet `n_est` stays at 32–47. The residue is fragments separated by **gaps** where Basic Pitch's frame threshold dropped out mid-wobble — and the pyin contour is continuously voiced straight across them.

Criterion: fuse two pitch-identical events across a gap when the contour is voiced through the whole gap (`voiced_prob` ≥ `PYIN_MIN_VOICED_PROB`, already at `:319`) and never leaves a semitone band. Thread the contour `_refine_pitches` already computes into the merge stage rather than calling `_pyin_contour` twice.

Put it behind a flag in `config.py` beside `f0_merge`/`pyin_semitone`, following the established pattern (a property, not a class attribute, so the harness can A/B it in-process). Note `MERGE_GAP_MELODY` is 0.08 s and these gaps are larger, so the ceiling must come from the contour, not from widening that constant.

Success: `n_est` on the 9 vibrato cases approaches 11, `ghost_count` falls, and the eight 117-BPM readings fix themselves as a *consequence* — once the median inter-onset interval is a real note length, `_grid_bpm` stops declining. Guard cases (`major_c`, `minor_harmonic_a`, `semitone_steps_c_major`, `vibrato_depth_0_8`) must not move; `vibrato_depth_0_8` matters most, being the one vibrato case already at `n_est == n_ref`.

1d. Instantiate the Basic Pitch model once

Hold a module-level `Model(ICASSP_2022_MODEL_PATH)` behind a lazy accessor and pass the instance to `predict` instead of the path (`transcriber.py:2047-2081`). Weights are already baked at image build. Throughput only — assert the accuracy report is byte-identical. This is what makes 1a affordable per job.

Phase 2 — make the result measurable (critical path for Phase 3)

With compute unconstrained, every Phase 3 decision becomes a *model-selection* decision, and model selection cannot be done on 38 synthetic sinusoid-vibrato clips. This phase is what turns Phase 3 from a guess into a measurement.

2a. Real recordings

`server/tests/accuracy/real/` has a pinned label format and **no loader and no recordings**. `runner.py` and `report.py` know only about `corpus.py`. The corpus README is explicit: it is "a regression instrument, not an accuracy claim", and absolute accuracy "must only ever be quoted from real recordings".

Write the loader to the format already specified in `real/README.md` — the beat conversion is spelled out there, and given `list[RefNote]` + `key` + `tempo_bpm` **the existing metric functions need no change**, which is why the format was pinned before the loader. Record 10–20 clips inside the stated constraints (monophonic, ≤ MIDI 84, notes ≥ ~120 ms), with consent, kept out of the repo unless cleared. Include clips longer than 60 s so 1a is measurable. Report real and synthetic aggregates **separately** — never merge them.

2b. Meter and metrical level

Nothing can be claimed about meter today: `metric_namespaces` is `['notes','beats','key','tempo','solfa','conf']` — there is **no `meter` namespace at all**, and `Case` has no `time_signature` field. (The `transcriber.py:2160-2198` post-mortem describes a `meter.exact` gate that was never committed.)

1. **Plumb the metrical level.** `:2195` names this as the real defect: "this function is not told which metrical level `bpm` is on." `_grid_bpm` already *knows* it — `_TEMPO_CANDIDATE_FACTORS` at `:1865` is the level it chose — so carry that out and through to the `TimeSignature` write at `:2198`.
2. **Score it.** Add `time_signature` to `Case` (`corpus.py:89-105`) and to the `real/` sidecar spec; add a `meter` namespace to `metrics.py` scoring exact match **and bar duration in seconds**. The post-mortem's sharpest point is that a correct "6/8" over a bar-rate tempo engraves a bar 2.9× too long while scoring `exact=True` — an exact-match-only metric would call that a pass.
3. **Add non-4/4 cases** (3/4, 6/8) to `synth.py` / `corpus.py`, which needs a meter concept in `RenderOptions` / `RefNote` — there is none today.

Re-baseline deliberately: adding cases changes `corpus.fingerprint` (currently `0146e7b5cb03b287`, `n_cases: 38`), so aggregates are not comparable across that boundary. Regenerate and commit the new baseline in its own commit.

Phase 3 — engine upgrade, chosen head-to-head

Torch-class models are now admissible. Adopt them where the pipeline has a *capability gap* or a measured shortfall — not where it is already strong.

3a. Segmentation bake-off (the defect)

Run candidate engines against the Phase 1 pipeline on the Phase 2 real corpus, scored by the existing `metrics.py`: note F1 with offset, `n_est` vs `n_ref`, `solfa.degree_accuracy` and `ghost_count`. Candidates from the document worth trying, all aimed at segmentation rather than pitch: MT3 / YourMT3+, Transkun, and singing-transcription models trained on vocal note events. Keep the current engine as the control.

Two disciplines from the strike record: adopt an engine only if it beats the Phase 1 pipeline **on real clips**, and keep the existing post-processing chain (`_merge_onsets`, `_resolve_melody_overlaps`, key detection, respelling, solfa) — a new front-end replaces inference, not the notation logic, and it must still emit something shaped like `Event` (`transcriber.py:1144-1185`) so `bend_semitones` / `pyin_midi` / `voiced_prob` survive.

3b. Neural downbeat and meter (the capability gap)

This is where a model is most clearly justified: both classical routes are struck on this material (`tempogram_ratio` anti-correlated, candidates ranked 13th of 40; the symbolic route 0 of 38), and meter is a real product feature currently hardcoded. Prefer a modern torch downbeat tracker over madmom, whose numpy/Python conflict confines it to an isolated sidecar. **Gate on 2b existing**, or the result cannot be scored.

3c. Ensemble the confidence signal

`conf.auc 0.676` and `conf.separation 0.032` are the weakest numbers in the baseline, against `key.confidence_auc 0.949`. Per-note confidence is what the UI needs to flag uncertain notes, and `_blend_confidence` (`:1530`) currently derives it from amplitude and pyin agreement. Cross-engine agreement (Phase 3a's candidates, or a second f0 tracker such as CREPE/PENN) is a genuinely new evidence source for it — this is the one place the document's pitch-ensemble idea earns its keep.

First confirm which number is current: the `pyin_semitone` docstring quotes `conf.auc 0.856 → 0.868`, which does not match the committed baseline's 0.676. Resolve that discrepancy before optimising against either. (The document's "0.926 AUC" citation refers to *key* confidence, not note confidence.)

3d. Source separation, gated

Evaluate Demucs only if Phase 2's real clips show accompaniment or room noise is a live failure mode. Cheap to run with compute available; still needs evidence before it enters the path.

3e. Fine-tuning, gated and validated on real audio only

The document's "train on 10k–100k synthetic mixtures" is now feasible, and that makes the overfitting risk sharper rather than smaller — this is the loop the four strikes came from. Permissible only as *augmentation* for a fine-tune whose validation set is real recordings, never synthetic. Do not attempt before 3a has established which engine is worth tuning.

Phase 4 — what accuracy needs from the new platform

- **GPU** for Phase 3a/3b inference; CPU-only torch would reintroduce exactly the latency pressure that created the `pyin_max_seconds` defect.
- **RAM** sized from a measured spike in the real image (`server/Dockerfile`; precedent `spike/.venv`) with torch plus the chosen model resident alongside onnxruntime, librosa and music21 — not from a guess.
- **Remove the single-worker constraints** that exist for CPU reasons only: `WEB_CONCURRENCY: 1`, `inference_workers: 1`, `pyin_max_seconds: 60`.
- **Keep unchanged**: audio deleted immediately after each job (TRD par. 9), the camelCase wire contract, and `ApiError {code, message, details?}`. Clients still talk only to the C# API; the worker stays internal.
- The transcriber deploys as a **container** (`heroku.yml` → `server/Dockerfile`), so the image is portable to another platform largely as-is; `Aptfile` deps (ffmpeg, libsndfile1) move into the Dockerfile if the new host has no apt-buildpack equivalent.
- Keep the **`basic-pitch --no-deps`** + onnxruntime arrangement until 3a actually replaces the engine; installing TensorFlow alongside torch buys nothing.

Verification

Per phase, in order. The accuracy suite runs only under `spike/.venv` — plain `python -m pytest` has no pytest on this machine.

```
cd server && ../spike/.venv/Scripts/python -m pytest -q
```

1. **Directional tests first.** `tests/accuracy/test_directional.py` is the established pattern for a decision chosen from data: it asserts on the *decision*, not the aggregate, because "a corpus metric cannot distinguish 'fused the right pair' from 'fused nothing and got lucky'". 1c needs one; 1b needs one asserting an out-of-band fallback is folded into the notatable band.
2. **Regenerate the accuracy report and diff against `baseline.json`.** The harness's 0.02 regression tolerance is the gate. 1b's predicted numbers are stated above — if `tempo.abs_error` does not land near 11.5, the diagnosis is wrong, not the tolerance.
3. **A/B each flag in-process**, which is what the `Settings` properties exist for: run 1c off then on in one process rather than trusting a single run.
4. **Guard cases must not move**: `major_c`, `minor_harmonic_a`, `semitone_steps_c_major`, `vibrato_depth_0_8`.
5. **1d is accuracy-neutral by definition** — assert a byte-identical report, and time a warm second request to confirm the throughput win.
6. **Phase 2 onward, quote real and synthetic separately.** Any accuracy claim leaving this project must come from `real/`.
7. **Full backend suite before any deploy**: `dotnet test NoteFusion.slnx` (650 passing today) plus the server suite above. Nothing in Phases 1–2 touches the wire contract, so `src/NoteFusion.Application` DTOs and `mobile/src/api/types.ts` need no change — if Phase 3c starts emitting extra confidence fields, both change together per CLAUDE.md.
8. **Solfa engine lock-step**: anything reaching `server/app/pipeline/solfa.py` must be mirrored in `src/NoteFusion.Core/Solfa` with unit tests. Phases 1–3 as scoped do not touch it.
9. **Deploy only after the regenerated report is committed**, so the baseline in git always matches the shipped code.